L4： 需要熟练使用脚本调用工具，实现复杂视觉算法搭建视觉系统，根据客户需求确定项目实施方案。

目录

考试大纲 2
一、Vision Tools： 2
1.C#简单脚本基础介绍 2
2.脚本调试。 2
3.工具调用及参数的输入输出。 2
4.VPP 界面显示(Inspection 输入输出的介绍)-参考以及机试操作 4
5.循环排序-具体详细介绍参考 C#基础文档 7
6.Help 的查询。 8
二、 Framework 介绍-参考 C#第二阶段培训文档-视频 8
1.框架介绍 8
2. Machine Support 各个模块的介绍：光源模块、通讯模块、Vpp 模块儿 14
3. Visual Studio 的调试方法。 14
4.Inspection 的添加 15
5.通讯协议的修改。 15
6.VPP 数据传递。 16
7.相机调用 16
8.标定模块的设置和添加。-参考第二阶段视频 17
9.设备映射逻辑介绍。-参考第二阶段视频 17
10.设备通讯逻辑介绍、-参考第二阶段视频 17
11.数据收集模块-参考第二阶段视频 17
12.跑图的设置。-参考第二阶段视频 17
三、Dataman 脚本调试与修改。 17
1.多码调试应用 17
2.选型 17
3.码密度计算 19
4. ID 脚本参考-3V 验证器评判标准 20
机器视觉维护与应用技能等级要求--工作领域 24
一、机器视觉系统方案评估 24
1. 相机选型 24
2. 镜头选型 25
3. 光源选型： 26
4.精度计算 27
3D 视觉系统图像处理 27
5.光源安装位置设计 28
6. 相机、镜头安装位置设计 28
7. 项目风险评估 28
8. 确认视觉方案文档 28
二、视觉系统集成与应用 29
1 视觉系统集成 29
2 视觉系统编程 29
3 视觉系统调试 29

考试大纲

一、Vision Tools：

1.C#简单脚本基础介绍

参考文件夹内 C#基础的 TXT 文档。

2.脚本调试。

参考机试操作

3.工具调用及参数的输入输出

参考 PMAlign 工具示例以及机试操作

① **设定运行参数：** RunParams

```
pma.RunParams.AcceptThreshold = 0.6;//运行参数界面 接受阈值
pma.RunParams.ApproximateNumberToFind = 3;//运行参数界面，查找概数。
pma.RunParams.AutoEdgeThresholdEnabled = true;//运行参数界面，自动边缘阈值。
pma.RunParams.EdgeThreshold = 20;//运行参数界面，自动边缘阈值。
pma.RunParams.CoarseAcceptThresholdEnabled = false;//运行参数界面，粗糙度接受阈值法启用/不启用
pma.RunParams.CoarseAcceptThreshold = 0.34;//运行参数界面，粗糙度接受阈值为 0.34
pma.RunParams.ContrastThreshold = 11;//运行参数界面，对比度阈值。
int n=pma.RunParams.ChangedEventSuspended;
pma.RunParams.GrainLimitsUsePattern = false;//运行参数界面的自动粒子模式限制开关。
pma.RunParams.GrainLimitFine = 1;//运行参数界面粒度参数下限  精细
pma.RunParams.GrainLimitCoarse = 2;//运行参数界面粒度参数上限 粗糙
pma.RunParams.HasChanged = true;
pma.RunParams.OutsideRegionThreshold = 0.1;//搜索区域界面下 的 区域以外计分参数，特征阈值
pma.RunParams.OwnedFlexParams.ControlPointsX = 6;
pma.RunParams.RunAlgorithm= CogPMAlignRunAlgorithmConstants.PatQuick;// 运行参数界面的几种算法选择.PatMaxPerspective.PatMax.PatFlex.BestTrained
pma.RunParams.RunMode= CogPMAlignRunModeConstants.SearchImage;// 运行参数界面模式： 搜索图像和.RefineStartPose
pma.RunParams.ScoreUsingClutter = false;//运行参数界面计分时考虑杂斑选项
//pma.RunParams.SearchImageMask;//获取搜索图像相关信息。
```

pma.RunParams.SearchRegionMode = CogRegionModeConstants.PixelAlignedBoundingBoxAdjustMask;//搜索区域下区域模式.AffineTransform.PixelAlignedBoundingBox

pma.RunParams.Timeout = 5000;//运行参数界面 超时

pma.RunParams.TimeoutEnabled = false;////运行参数界面 超时选项

pma.RunParams.XYOverlap = 0.7;//运行参数界面 XY 重叠参数。

pma.RunParams.ZoneAngle.High=0.6;//单位弧度运行参数界面，角度缩放相关：.Overlap 重叠.Nominal 标称值.Low 下限.High 上限

pma.RunParams.ZoneScale.High=1.2;//运行参数界面，整体缩放相关：.Overlap 重叠.Nominal 标称值.Low 下限.High 上限

pma.RunParams.ZoneScaleX.High=1.2;//运行参数界面...

pma.RunParams.ZoneScaleY.High=1.2;//运行参数界面...

② 设定训练参数：Pattern

pma.Pattern.AutoEdgeThresholdEnabled = false;//训练参数里自动边缘阈值开关。

pma.Pattern.EdgeThreshold = 11;//训练参数里自动边缘阈值取值设定。

pma.Pattern.Elasticity = 0.1;//训练参数里弹性参数取值设定。

pma.Pattern.ExpectedDeformationRate = 0.5;//取值 0-1 不清楚是什么参数。

pma.Pattern.GrainLimitAutoSelect = false;//训练参数界面的自动粒子模式限制开关。

pma.Pattern.GrainLimitFine = 1;//训练参数界面粒度参数下限 精细

pma.Pattern.GrainLimitCoarse = 2;//训练参数界面粒度参数上限 粗糙

pma.Pattern.IgnorePolarity = false;//训练参数下忽略极性开关。

pma.Pattern.HighSensitivityParameter = 1; //训练参数下高灵敏度参数。取值范围 1-10

pma.Pattern.RepeatingPatterns = false;//训练参数下重复模式

pma.Pattern.TrainAlgorithm= CogPMAlignTrainAlgorithmConstants.PatQuick;// 训练参数下训练的算法.PatMaxPerspective.PatMaxHighSensitivity.PatMaxAndPatQuick.PatMax.PatFlex

pma.Pattern.TrainMode= CogPMAlignTrainModeConstants.Image;// 三种训练模式.ShapeModelsWithTransform;.ShapeModelsWithImage.Image

pma.Pattern.TrainRegionMode= CogRegionModeConstants.PixelAlignedBoundingBoxAdjustMask;// 训练区域模式.PixelAlignedBoundingBox.AffineTransform

③ 搜索区域：SearchRegion

//eg：一般 会搭配循环使用，比如设定固定的搜索区域一次搜索一个区域。通过几行几列把所有区域都搜索过来一遍。

```
CogRectangle rect2 = new CogRectangle();
for (int i = 0; i < 5; i++)
{
  for (int j = 0; j < 5; j++)
  {
    //起点+长宽方式创建矩形
     rect2.SetXYWidthHeight(0+100*j,0+i*100, 100, 80);//先水平方向平移 4 次一次 100。然后到第二行，再水平平移 100 距离四次，共计 25 个搜索区域。
    pma.SearchRegion = rect2;
    pma.Run();
    System.Threading.Thread.Sleep(30);//可以加个休眠，算是延时。可以帮助看 25 个位置是否合适。 单位 ms
  }
}
CogRectangle rect = new CogRectangle();
rect.SetCenterWidthHeight(150, 150, 150, 130);//中心+长宽方式创建矩形
pma.SearchRegion = rect;
```

④ 拿结果：Results

```
pma.Run();
    int pmacount=pma.Results.Count;//获取结果数量。
  double n1= pma.Results[0].Clutter;//杂斑
   double n2= pma.Results[0].Contrast;//对比度
   double n3= pma.Results[0].Coverage;//范围
   bool n4= pma.Results[0].FineStage;//精细阶段
   double n5= pma.Results[0].FitError;//拟合误差
   double n6= pma.Results[0].ID;//索引 序号
   double n7= pma.Results[0].MaxCoarseAcceptThreshold;//最大粗接受阈值
  double n10= pma.Results[0].OutsideRegionAreaProportion;//外部区域占比
    double n11= pma.Results[0].OutsideRegionFeatureProportion;//外部区域特征。
    double n12= pma.Results[0].Score;//第 0 项结果的分数。
  double d1= pma.Results[0].GetPose().Rotation;//第 0 项结果的弧度
   double d2= pma.Results[0].GetPose().Scaling;//第 0 项结果的整体缩放
   double d3= pma.Results[0].GetPose().ScalingX;//第 0 项结果的 X 缩放
   double d4= pma.Results[0].GetPose().ScalingY;//第 0 项结果的 Y 缩放
   double d5= pma.Results[0].GetPose().Skew;//第 0 项结果的倾斜
   double d6= pma.Results[0].GetPose().TranslationX;//第 0 项结果的坐标 X
       double d7= pma.Results[0].GetPose().TranslationY;//第 0 项结果的坐标 Y
```

4.VPP 界面显示(Inspection 输入输出的介绍)-参考以及机试操作

CogToolBlock1 脚本

文件(F) 编辑(E) 搜索(S) 脚本(C) 生成(B)

发行

CogToolBlockAdvancedScript | GroupRun(ref string message, ref CogToolResultConstants

```
1   namespace imports
11  public class CogToolBlockAdvancedScript : CogToolBlockAdvancedScriptBase
12  {
13    #region Private Member Variables
14    private Cognex.VisionPro.ToolBlock.CogToolBlock mToolBlock;
15
16    ArrayList list = new ArrayList();
17
18    #endregion
19
20    /// <summary>
21    /// Called when the parent tool is run.
22    /// Add code here to customize or replace the normal run behavior.
23    /// </summary>
24    /// <param name="message">Sets the Message in the tool's RunStatus.</param>
25    /// <param name="result">Sets the Result in the tool's RunStatus</param>
26    /// <returns>True if the tool should run normally,
27    ///          False if GroupRun customizes run behavior</returns>
28    public override bool GroupRun(ref string message, ref CogToolResultConstants result)
29    {
30      // To let the execution stop in this script when a debugger is attached, uncomment the fol
31      // #if DEBUG
32      // if (System.Diagnostics.Debugger.IsAttached) System.Diagnostics.Debugger.Break();
33      // #endif
34      // Run each tool using the RunTool function
35      foreach(ICogTool tool in mToolBlock.Tools)
36        mToolBlock.RunTool(tool, ref message, ref result);
37      list.Clear();
38      CogGraphicLabel label = new CogGraphicLabel();
39      label.SetXYText(1, 1, "Hello");
40      list.Add(label);
41
42
43      return false;
44    }
45
46    When the Current Run Record is Created
56
57    #region When the Last Run Record is Created
58    /// <summary>
59    /// Called when the last run record may have changed and is being reconstructed
60    /// </summary>
61    /// <param name="lastRecord">
62    /// The new last run record is available to be initialized or customized.</param>
63    public override void ModifyLastRunRecord(Cognex.VisionPro.ICogRecord lastRecord)
64    {
65      foreach (ICogGraphic a in list)
66      {
67        mToolBlock.AddGraphicToRunRecord(a, lastRecord, "CogIPOneImageTool1.InputImage", "");
68      }
69
70    }
71    #endregment
72
```

像ArrayList、List<CogLine>、CogGraphicCollection、都写在这里

脚本编辑区域

显示写在这里

① **常规完整显示方法：**

先写存储显示内容的集合()：比如以下几种方式

ArrayList list=new ArrayList();//存文本、存圆、线、等等。 泛型集合
List<CogLine> list_line=new List<CogLine>();//存线
CogGraphicCollection gc=new CogGraphicCollection();//存文本、存圆、线、等等。
后边一般要跟上清除/清空。比如：
list.Clear(); list_line.Clear(); gc.Clear();
然后可以写文本、圆、线等要显示的对象及添加进提前写好的存储显示内容的集合中：
CogGraphicLabel label = new CogGraphicLabel(); //new 个存文本的对象，
label.Alignment = CogGraphicLabelAlignmentConstants.TopLeft;//指定存储位置为设定 XY 的某个比如左上、中心等位置。
label.Color = CogColorConstants.Red; //指定字体颜色
label.SetXYText(x, y, s);//指定要显示的位置以及内容。
list.Add(label);//把写好的这个文本对象加进提前写好的集合中。
然后显示出来：同样的可以写显示文本、圆、线等。也可以全写一起(建议用这一种。)
显示文本：
foreach (CogGraphicLabela a in list)//只可以显示 List 里文本部分，前提是 List 里放的有文本。
  {
    mToolBlock.AddGraphicToRunRecord(a, lastRecord, "CogIPOneImageTool1.InputImage", "");
      }
foreach (CogCircle a in list)//只可以显示 List 里圆部分，前提是 List 里放的有圆。
  {
    mToolBlock.AddGraphicToRunRecord(a, lastRecord, "CogIPOneImageTool1.InputImage", "");
      }
foreach (CogLine a in list)//只可以显示 List 里圆部分，前提是 List 里放的有线。
  {
    mToolBlock.AddGraphicToRunRecord(a, lastRecord, "CogIPOneImageTool1.InputImage", "");
      }
foreach (ICogGraphic a in list)**//全显示，不需要考虑里边放了什么**
  {
    mToolBlock.AddGraphicToRunRecord(a, lastRecord, "CogIPOneImageTool1.InputImage", "");
      }

② 灵活运用：
CogGraphicLabel myLabel = new CogGraphicLabel();//模板(放上多个文本中相同的，不需要再次修改的内容，比如多个文本虽有不同，但是都显示在同一个空间、比如字号都相同、比如背景色始终相同。)
CogGraphicLabel myLabelTmp = new CogGraphicLabel();//这个是根据实际效果，可以先复制前边的模版，省去多个文本相同部分的脚本编辑，然后再自定义其它不同的脚本(颜色、字体等)。

//这里定义文本模板： 比如都显示在 #空间下，颜色都是绿色字体，文本是宋体 字号 30
myLabel.Alignment = CogGraphicLabelAlignmentConstants.BaselineLeft;
myLabel.SelectedSpaceName = "#";
myLabel.Color = CogColorConstants.Green;
myLabel.Font = new Font(“宋体”,30);

//这里是具体的某个文本，先复制模板内容，然后添加进显示具体位置，然后存进集合。
myLabelTmp = myLabel.Copy(CogCopyShapeConstants.All);
myLabelTmp.SetXYText(x_Init, y_Init + y_step * nRow++, "DeltaXTop= " + DeltaXTop.ToString("f3"));
labelList.Add(myLabelTmp);
参考 VisionPro 文档：CogGraphicLabel 属性界面：

③ 相对高级应用(最实用)：尤其是需要多个显示，以及需要根据某些条件来判定显示 OKNG/绿色红色等。
public void Graphic(double x, double y, string s, bool b)

//自定义显示方法需要传显示位置 XY，显示文本 s，根据判定条件拿到的 ture 或者 false

```
{
  CogGraphicLabel label = new CogGraphicLabel();
  label.Alignment = CogGraphicLabelAlignmentConstants.TopLeft;
  label.Color = b == false ? CogColorConstants.Red : CogColorConstants.Green; //根据 true 和 false 对应绿色红色
  label.SetXYText(x, y, s + ( b == false ? "NG" : "OK"));//根据 true 和 false 对应显示 OKNG
  list.Add(label);
}
```

调用：Graphic(0, 100, "中间弹片： ", false); 根据自定义的方法提供需要的 xy 文本，true 或者 false 条件。

// X Y S 文本 ds 是比如拿到的距离，个数等。 d 是 判定条件，(比如要求个数小于 5，距离大于 100 等)

```
public void Graphic(double x, double y, string s, double ds, double d)
{
  CogGraphicLabel label = new CogGraphicLabel();
  label.Alignment = CogGraphicLabelAlignmentConstants.TopLeft;
  label.Color = ds < d ? CogColorConstants.Red : CogColorConstants.Green;
  label.SetXYText(x, y, ds < d ? "NG: " + s : "OK: " + s);
     if(d == 0.3 || d == 1)
     //根据不同的判定条件，有可能会需要显示不同的效果，比如要求大于多少才 OK 且绿色，或者小于多少才 OK 且绿色
  {
    label.SetXYText(x, y, ds > d ? s + "NG" : s + "OK");
    label.Color = ds > d ? CogColorConstants.Red : CogColorConstants.Green;
  }
  list.Add(label);
}
```

调用：Graphic(0, 25, "外侧圆夹角： ", angle2, 1)根据自定义的方法提供需要的 x、y、文本，判定值，条件。

④ 只保留一个图层：

```
ICogRecord hold = lastRecord.SubRecords["CogIPOneImageTool1.OutputImage"];//保存需要保留的图层
  lastRecord.SubRecords.Clear();//清除当前全部图层。
  lastRecord.SubRecords.Add(hold);//添加需要保留的图层。
  foreach(ICogGraphic label in gc)//foreach 循环方式把需要显示的内容显示在指定图层。
  {
    mToolBlock.AddGraphicToRunRecord(label,lastRecord,"CogIPOneImageTool1.OutputImage","");
      }
```

⑤ VisionPro 文档 CogGraphicLabel 的属性界面。

Name	Description
Alignment	Controls how the text is aligned with respect to the X,Y position.
BackgroundColor	Controls the background color for text. Use cogColorNone for transparent.
ChangedEventSuspended	If nonzero, indicates that the raising of the Changed event has been suspended. This value is incremented when SuspendChangedEvent is called and decremented when ResumeAndRaiseChangedEvent is called.
Children	Children of this graphic.
Color	Color of this graphic. Can be any OLE_COLOR.
DragBackgroundColor	Controls the background color for text when the label is being dragged. Use cogColorNone for transparent.
DragColor	Color of this graphic when dragged.
DragLineStyle	Line style of this graphic when dragged.
DragLineWidthInScreenPixels	Line width of this graphic when dragged.
Font	
GraphicDOFEnable	Interactive Degree of freedom for this graphic object. Provides the ability to govern interactive manipulation of an object; for example could be used to prevent an object from being resized.
GraphicDOFEnableBase	Interactive Degree of freedom for a graphic object. Allows access to each graphic's GraphicDOFEnable property without knowing the specific graphic type.
HasChanged	If true, the serializable state of this object has changed since the last time it was serialized.
Interactive	Interactive property allow this graphic object to be selected. See GraphicDOFEnable for information on governing interactive manipulation of an object.
LineStyle	Line style of this graphic.
LineWidthInScreenPixels	Line width (in screen pixels) of this graphic.
MouseCursor	Mouse cursor to be displayed when the mouse is over the graphic. The cursor will be visible only if the graphic is interactive.
Offset	The distance from X,Y, as measured in OffsetSpaceName coordinates,to use to adjust the label position.
OffsetAngle	The angle (in radians) as measured in OffsetAngleSpaceName coordinates to use to adjust the label position.
OffsetAngleSpaceName	Coordinate space in which this shape's OffsetAngle property is to be interpreted.
OffsetSpaceName	Coordinate space in which this shape's Offset property is to be interpreted.
Parent	Parent of this graphic.
Rotation	The angle (in radians, from the x-axis) of the text label.
Selected	True when the shape is selected in a display.
SelectedBackgroundColor	Controls the background color for text when the label is selected. Use cogColorNone for transparent.
SelectedColor	Color of this graphic when selected.
SelectedLineStyle	Line style of this graphic when selected.
SelectedLineWidthInScreenPixels	Line width of this graphic when selected.
SelectedSpaceName	Coordinate space in which this shape is to be interpreted.
Text	Text for this label.
TipText	Text to describe this graphic in a tool tip.
Visible	Visible property of this graphic.
X	X coordinate of the text.
Y	Y coordinate of the text.

5.循环排序-具体详细介绍参考 C#基础文档

冒泡排序：

```
int[] number = { 2, 6, -43, -32, 77, -37, 44, 45, 23, -62, 33, 22 }; //要求降序排列 左边大 右边小

        for (int i = 0; i < number.Length - 1; i++)
        {
            for (int j = 0; j < number.Length - 1 - i; j++)
            {
                if (number[j] < number[j + 1])
                {
                    int temp = number[j + 1];
                    number[j + 1] = number[j];
                    number[j] = temp;
                }
            }
        }
        for (int i = 0; i < number.Length; i++)
        {
            Console.Write(number[i]+"  ");
        }
```

```
Console.ReadKey();
```

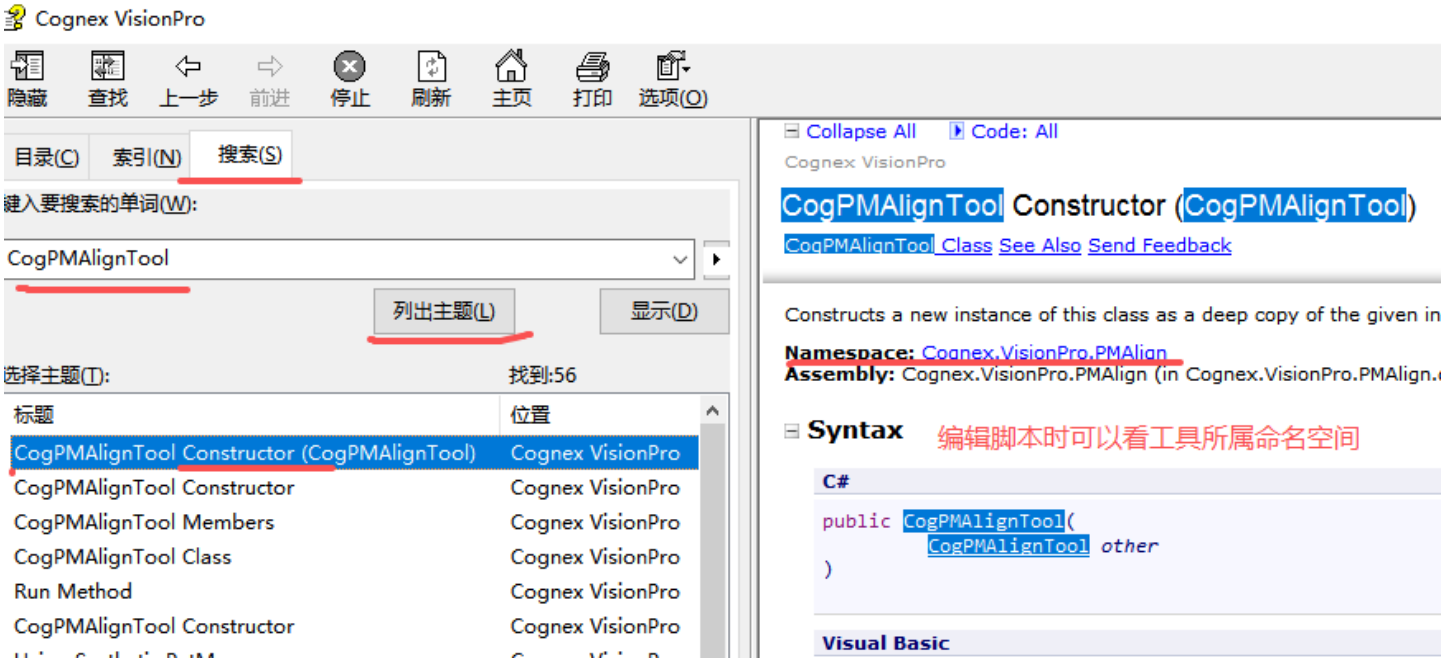
冒泡排序和选择排序：可以直接复制到 VS 中查看的：

```
//冒泡排序思路：通过俩俩比较，把最大/小值放在最右侧， 例如长为 7 的数组
//外循环次数：因为是俩俩比较，所以按照索引来是 0 1 比较，1 2 比较，最后是 5 6 比较。
//那么 i<长度-1 也就是 i<7-1  此时 i 取值为 0 1 2 3 4 5
//内循环，每次都需要，第 0 和 1 比较，然后 1 和 2 比较，但是要跳过最右侧已经拿到的最大/小值。
//所以内循环 j 需要取值为原有的基础减去 i 的值，i 为 0 还没比较那不需要，i 为 1 说明已经拿到了 1 个最大/小值，所以比较时 j 取值少取最后一位
//每次 j 的 0 和 1 比较时候若满足小/大条件，就直接先交换位置。通过 temp 第三方变量。
int[] nums = { 5, 9, 10, -110, 55, 123, 1 };//长 7
for (int i = 0; i < nums.Length-1; i++)//长 7 索引 i 应取 0 1 2 3 4 5
{
    for (int j = 0; j < nums.Length-1-i; j++)//已经比较过的值往右挪了，所以减去 i，让 j 随着 i 的循环比较次数变少
    {
        if (nums[j] < nums[j + 1])//左比右小，交换，那么就是小值放最右，从大到小排序
        {
            int temp = nums[j];//交换值。
            nums[j] = nums[j + 1];
            nums[j + 1] = temp;
        }
    }
}
foreach (int n in nums)
{
    Console.Write(n+"  ");
}
Console.ReadKey();
Console.WriteLine();

//选择排序思路：从数组里选第一位当最小/大看待，然后和后边每一位比较看他是否是最小/大，
//不是则把实际最小/大的索引给到 num 这个变量，然后嵌套的循环比较完每一个值之后交换最小/大值放在 i 位值。
//以此方法把值从小到大，或者从大到小，从左到右挨着拿到。
//外层循环 i 次数要＜长度－1 因为例如长 7，i 取值为 0 1 2 3 4 5.
//内层循环 j：起始位置为 i+1，每次都从 i 后 1 为开始比较，比较到最后一位，长 7，最后一位索引应是 6
//每次嵌套循环之后需要把拿到的最小/大的索引值和数组里 i 位置交换。
int[] nums2 = { 15, -9, 10, 110 , 55, -123, -1};//长为 7
for (int i = 0; i < nums2.Length-1; i++)//i 为长度－1
{
    int num = i;//默认 i 为最小
    for (int j = i+1; j < nums2.Length; j++)//
    {
        if (nums2[num] < nums2[j])//第一位和后续每一个比较，拿到最小所在索引
        {
            num = j;//把最小索引赋值给 num
        }
    }
    int temp = nums2[num];//通过索引拿到最小值，给到 i 所在的位置。
    nums2[num] = nums2[i];
    nums2[i] = temp;
}
```

```
foreach (int n in nums2)
{
    Console.Write(n+"  ");
}

Console.ReadKey();
```

6.Help 的查询。

在任意工具、Cogjob、CogToolBlock、Quickbuild 界面都可以点击？按钮，点开文档主页。
输入关键词：例如：
输入 CogPMAlignTool



输入 PMAlign 、blob、caliper、Histogram、等等搜索，找到 edit Control 选项，可以看中文工具使用详细介绍。

查看工具使用介绍：
添加工具，在工具窗口，点问号，然后点控制参考页-会自动弹出搜索界面，默认搜索了 edit Control，可以直达工具中文使用介绍界面

二、Framework 介绍-参考 C#第二阶段培训文档-视频

1.框架介绍

①.软件图标： 准星图标贴装、放大镜+准星贴装带检测、放大镜单检测(cami)

②.FrameWork： 是个视觉服务器，是 visionPro、机械手对位和检测设备之间的桥梁

主要包括内容：获取图像、自动标定、运行 INspection 的 ToolBlocks

与光源控制器对接、设置跟机台主控程序通讯的 TCP/IP 或者串口

显示 log 信息、呈现 inspection 结果给用户、保存图片、offline 模拟

③.界面介绍：

相机设置： 相机类型：Gige(2D)、Displacementsensor(3D 位移传感器)、Other

IP、带宽：满载 100%、亮度(增益)、对比度、限时、触发模式、触发极性

标定设置：

相机运动：Passthrough：系统不进行标定或者不在 calibration 内标定，为采集的图像已经含有标定信息的传感器或其它采图设备使用、

StaticPose：相机静止，常规 9/11 点标定等，Chardboard 标定。

MovingCameraXYTheta：标定块不动。相机运动标定。

MovingCameraXYThetaInverse：建立相对坐标系而使用

畸变模型：Physical Configuration：默认 0

Distortion Mode：扭曲模式，此项一般选择 OneParamRadial 模式，如果是真远心镜头可以选择 Telecentric 模式。

Degrees of freedom：角度自由度，分为 RotationAndTransLation(XYTheta)，

RotationOnly(OnlyTheta),TranslaionOnly1Axis(Only1Axis),TranslationOnly2Axies(OnlyXY),

11 点标定使用 RotationAndTranslation，9 点标定使用 TranslationOnly2Axies. 其它两项很少使用。

Calibration Plate Mylar 标定板规格：，按照实际使用填写，否则导致标定坐标系异常。

Minimum Rotation：最小旋转角度，最小 5°，否则计算不准确。

Align Home2D to Motion2D at Image Center:默认不勾选。

批处理：运行模拟文件，可以加载标定图片文件。 IDB editor 可以打开 idb、cdb 图片、打开标定完生成的文件。

Image 选项：image Attributes：包含 calibrationID、inputID、FovXY、RobotXY 等

Inspection 设置：

Dwell time：延时等多久再运行。

Lighting Dwell time：让光源持续亮几秒

Time out：inspection 运行时间超出设定的这个时间，就会运行失败。默认 10000ms

work Mode：Normal：vpp 会以系统框架认为的最快速度去运行。

sequence(单线程)：默认第一张图 run，run 不完，则等 10 次一次 10ms 完了还没 run 完就不等。直接 run 第二张图。

Dualcore 双线程：

lgnore load Toolblock：勾选表示忽略，意味下次开软件，不显示加载对应的 inspection 了，打开速度块，省空间

标定信息：上下限 新框架可以直接双击修改。

Fovxy、MotionAspect：机械手 Y/X 的值。MotionScalingX/Y=aspect 的值。

MotionSkew：模组/机械手的 XY 夹角。CalpiatePitchXY 标定片尺寸。

CameraRotationXY：当前相机安装方向和机械手方向的夹角。CenterXY，视野中心坐标

ImageSizeXY 相机分辨率。MotionCapability：畸变模型(标定设置里选择的)

PixAspectRatio：

PixelSizeXY：像素分辨率。TrainedSteps：拍次数(1 11 12 等)

④Tools 工具： Motion Error Visualizer：可以加载标定好的 vpp 查看标定结果怎么样。主要包含：XYA 误差、纵横比、标定图像点数的畸变程度。

最后一项可以加载两个标定文件，比如第一次标定和第二次标定，对比两次的误差。

可以查看，第几次拍照、input 坐标、期望坐标。实际坐标

将所有工具块设为发布(relesse)模式：自动将各个 inspection 里脚本里调试改发布

其余详细见图片：

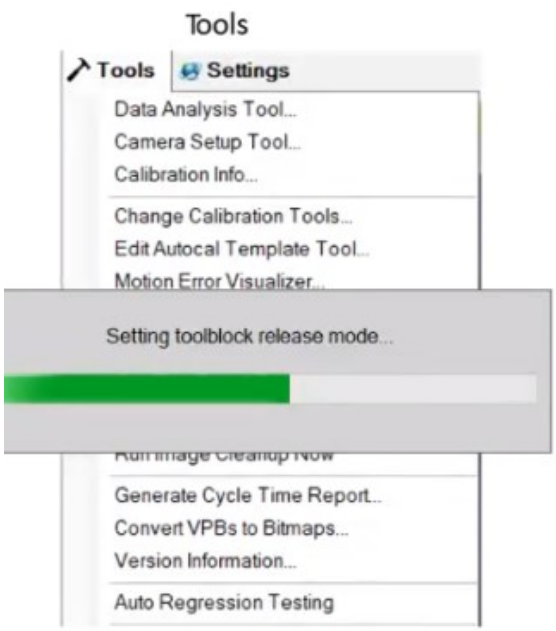
settings 设置:图像文件名字设置、添加不同权限的用户、视觉系统工具、

框架设置：窗口数量、界面分布、默认语言、窗口尺寸、最后图片尺寸、缓存大小、最多存多少图片

Asynthronous Communication Mode：异步通讯启用。

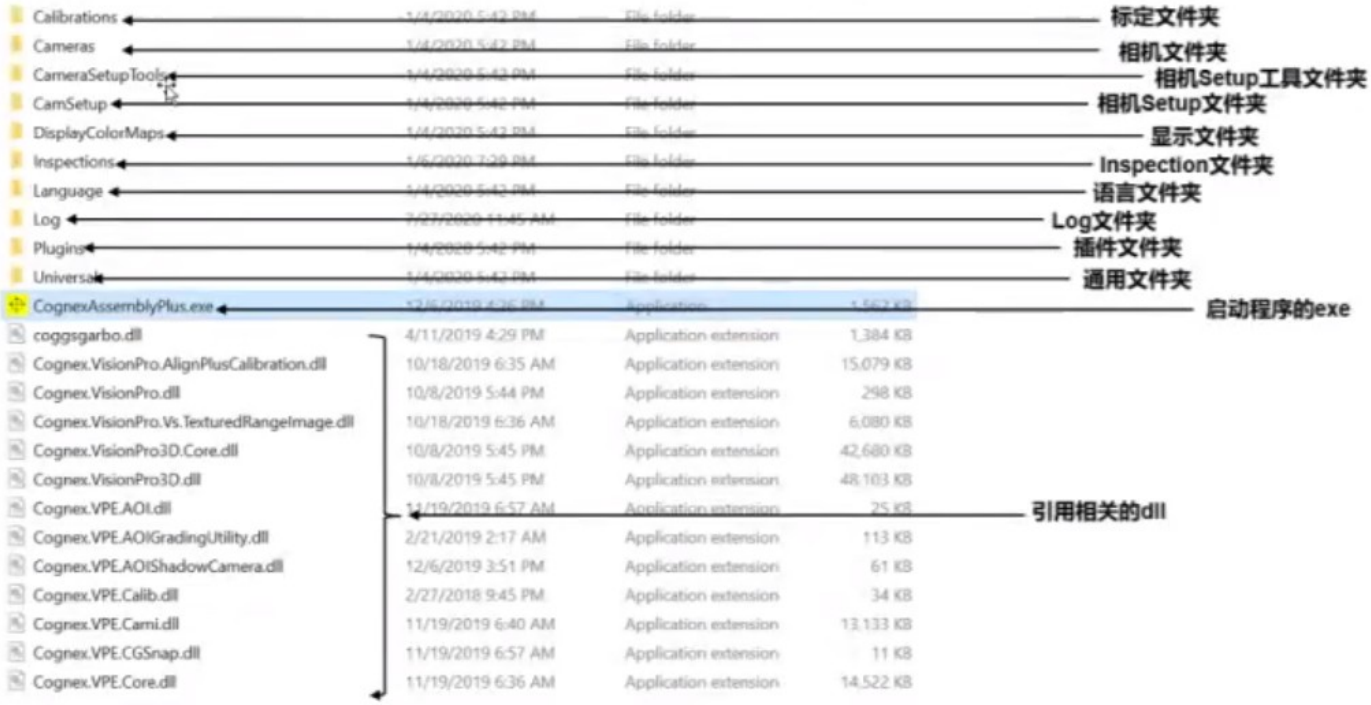
Listen for tcp client heartbeat:隔一段时间就确定一次通讯是否正常

Development Mode 开发者模式，非开发者模式 15 分钟上锁。

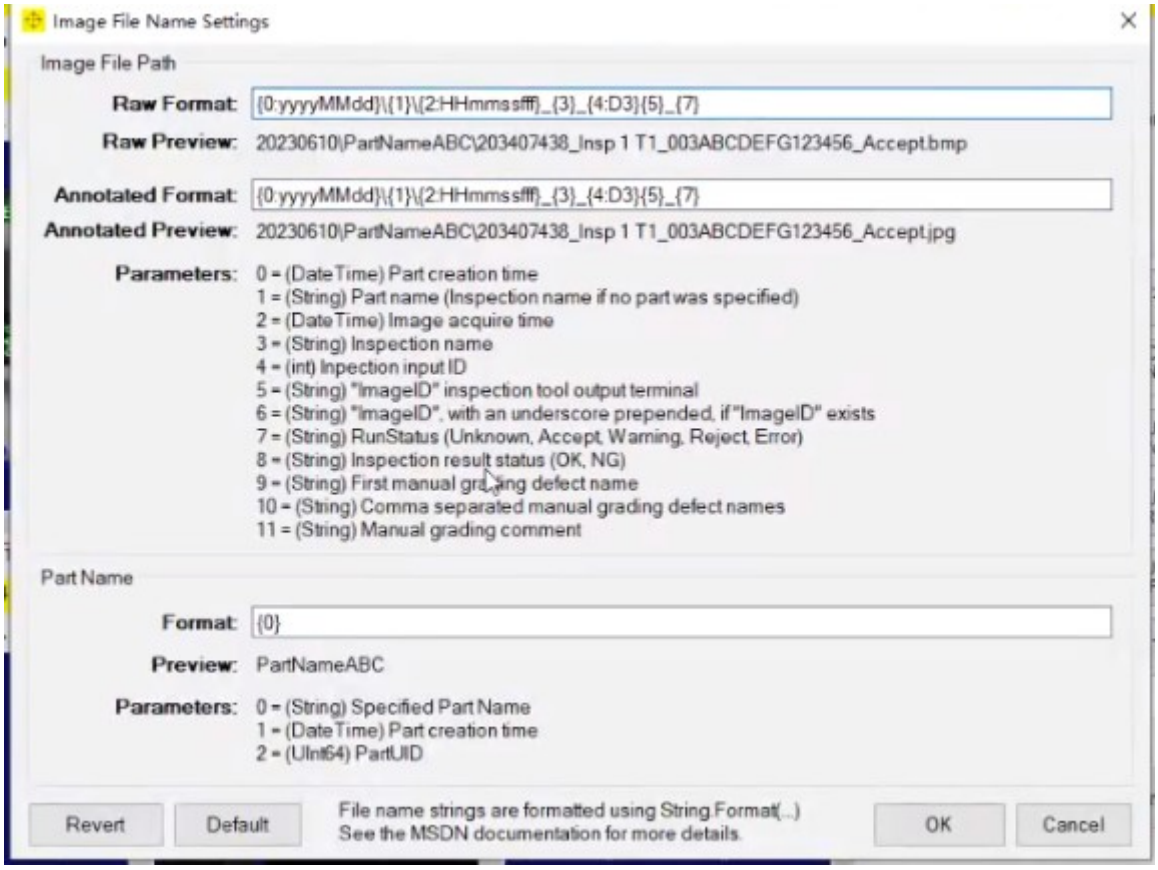


- Data Analysis Tool:数据分析工具，添加csv文件进行数据分析
- Camera Setup Tool:相机设置工具，MP架线保持相机的一致性
- Calibration info:可以查看标定结果信息
- Change Calibration Tools:更改标定工具(*)
- Edit Autocal Template Tool:更改自动标定工具(*)
- Motion ErrorVisualizer:分析机构误差
- Reset Autocal Calibration Scripts:还原自动标定脚本(*)
- Force All Tools to Release Mode:强制所有工具到Release模式
- Reduce Inspection ToolSize:减小Inspection工具尺寸
- Reload All Inspection:重新加载所有Inspection
- RunImage Cleanup Now:清理图片
- Generate Cycle Time Report : CT报告(+ machinesupport实现)
- Convert VPBs to Bitmaps:转换VPB(CGA -> VPB)文件为Bitmap
- Version information:版本信息
- Auto Regression Testing:自动回归测试

Framework 文件分布如下:

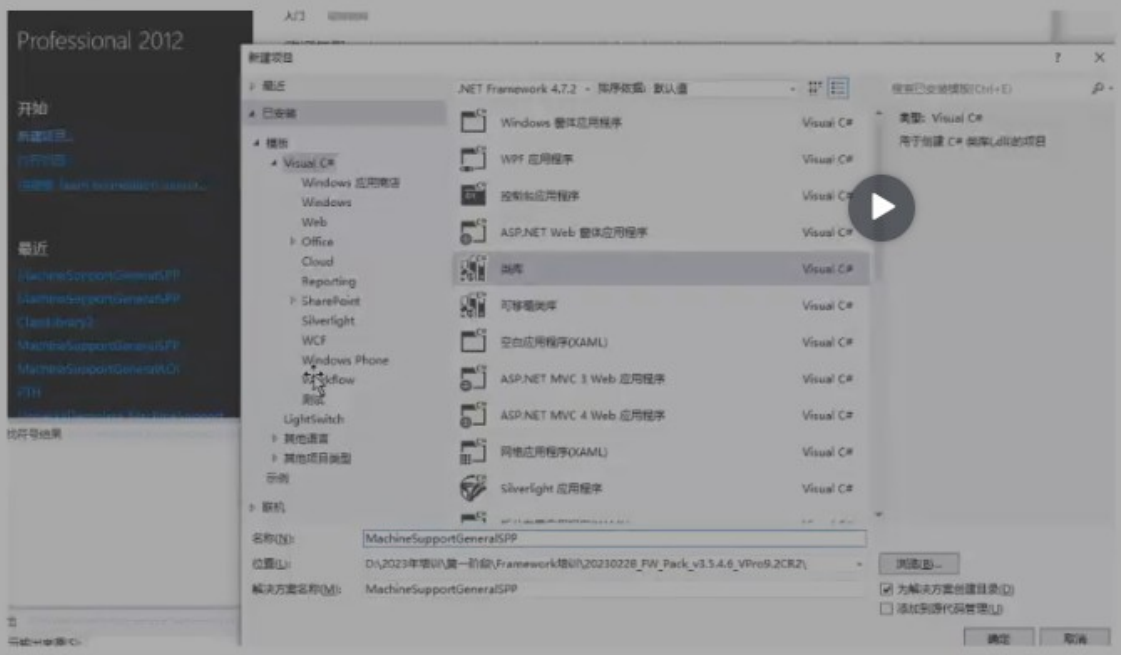


⑤ 存图名字设定： 参考图片



⑥ 创建一个新的框架及 MachineSupport：

3.打开VS2012，新建一个C#下的类库文件，重新命名MachineSupportGeneralSPP



创建一个新的MachineSupport

4.对这个类也重命名为“MachineSupportGeneralSPP”

5.添加VisionPro、Utility等相关引用，并引用对应using指令，将namespace 和程序集名称都修改为Cognex.VS.MachineSupport

COGNEX

创建一个新的MachineSupport

6.让MachineSupportGeneralSPP继承IMachineSupport并实现接口，并添加工厂类

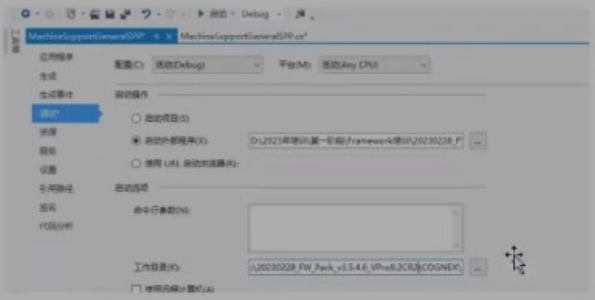
```
using Cognex.VS.MachineSupport;
namespace Cognex.VS.MachineSupport
{
    public class MachineSupportFactory
    {
        public static IMachineSupport Create()
        {
            return new MachineSupportGeneralSPP();
        }
    }
    public class MachineSupportGeneralSPP:IMachineSupport
    {

        public void ForceRecordFlush()
        {
            throw new NotImplementedException();
        }

        public CogGraphicCollection GetAcquisitionSetupOverlayGraphics
        {
            throw new NotImplementedException();
        }

        public string ImageSavedToDisk(bool savedToDisk, string savedT
        {
            throw new NotImplementedException();
        }

        public void Init(IFrameworkSupport framework)
        {
            throw new NotImplementedException();
        }

        public string MachineName
        {
            get { throw new NotImplementedException(); }
        }
```

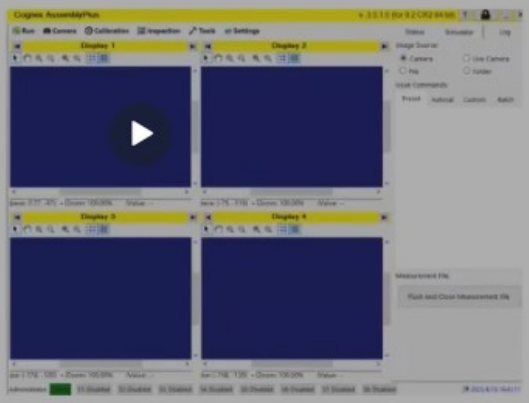
```
using System.Text;
using System.Threading.Tasks;
using Cognex.VisionPro;
using Cognex.VS;
using Cognex.VS.Utility;
using Cognex.VS.MachineSupport;
namespace Cognex.VS.MachineSupport
{
    public class MachineSupportFactory
    {
        public static IMachineSupport Create()
        {
            return new MachineSupportGeneralSPP();
        }
    }
    public class MachineSupportGeneralSPP:IMachineSupport
    {
        private IFrameworkSupport m_framework;
        string IMachineSupport.MachineName
        {
            get
            {
                return "SPP";
            }
        }
        public void ForceRecordFlush()
        {

        }

        public CogGraphicCollection GetAcquisitionSetupOverlayGraphics(int inspectionID, int inputID, int cameraID, int pr
        {
            return null;
        }

        public string ImageSavedToDisk(bool savedToDisk, string savedToDiskPath, bool movedOnce, ref CommandDestination de
        {
            return null;
        }

        public void Init(IFrameworkSupport framework)
        {
            return ;
```


© 2019 Cognex Confidential

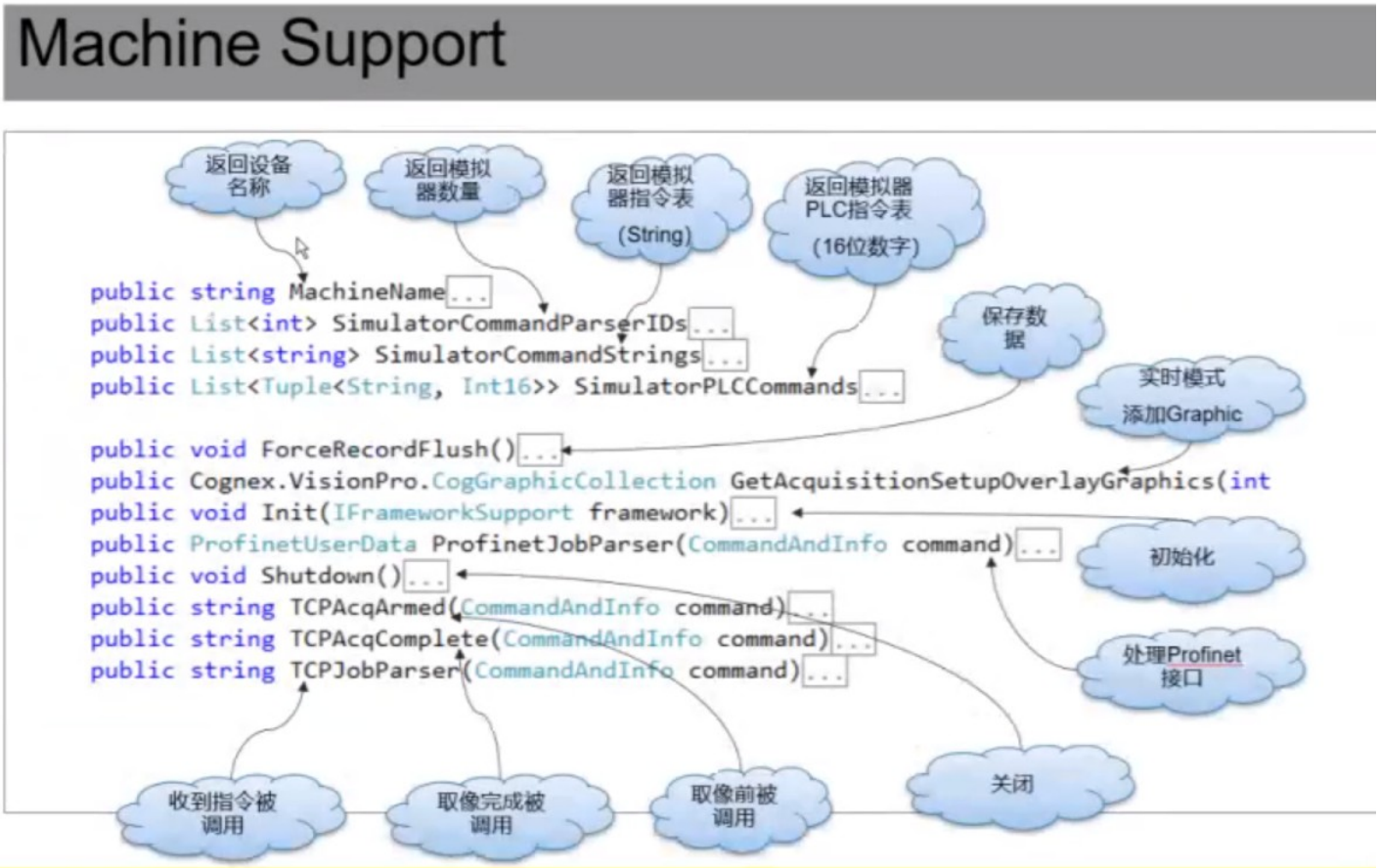
创建一个新的MachineSupport

7.更改调试和生成路径（外部程序、工作目录和输出路径）并生成解决方案，这个时候我们得到一个空的Framework程序





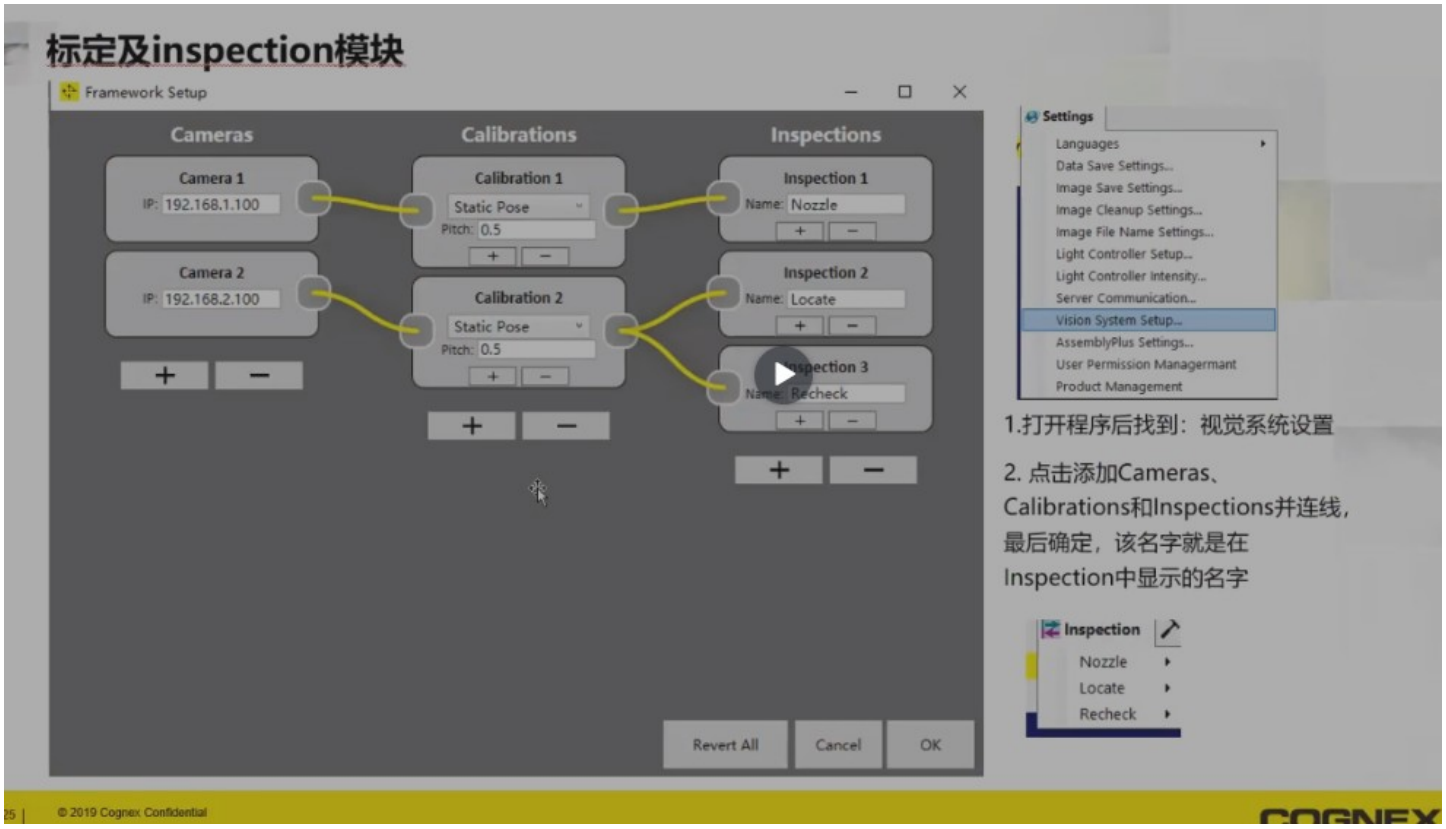
2. Machine Support 各个模块的介绍：光源模块、通讯模块、Vpp 模块儿



3. Visual Studio 的调试方法。

打开 MachineSupport 文件夹，打开 MachineSupport..... .sln 文件。会默认用 VS 打开。
VS 界面点调试，点 MachineSupport....调试属性。
属性界面：选择应用程序选型，目标框架选择 4.7.2.
调试选项：启动外部程序-选择浏览，找到软件目录下 Cognex 文件夹名为 CognexAssembly....exe 程序
工作目录选择，软件目录下 Cognex 层级。
选中 VS 界面右侧解决方案管理器 查看 Cs 类文件：MachineSupport.cs、等等
编辑完后点 VS 界面上方生成按钮，点重新生成解决方案，等待生成后若是没有报错即生成成功，报错了话就查找报错的地方，修改后再生成，警告可以忽略。

4.Inspection 的添加



5.通讯协议的修改。

TCPJob 函数的使用-参考视频

6.VPP 数据传递。

Machine Support：和VPP进行数据交换和传递

将数据输入到vpp的Input中

eSupport.HACTemplate_MachineSupport

Function_FinalInspection(CommandAndInfo command, int nDispl

```
CogTransform2DLinear MotionB_Pose = new CogTransform2DLinear();
MotionB_Pose.TranslationX = double.Parse(commandParts[i + 1])
MotionB_Pose.TranslationY = double.Parse(commandParts[i + 2])
MotionB_Pose.Rotation = CogMisc.DegToRad(double.Parse(command
m_Cave_Pose_Info[j].Camera_Pose = MotionB_Pose;
m_Cave_Pose_Info[j].CaveID = Cave_ID;
}

for (int i = 0; i < Cave_Num; i++)
{
    FrameworkConfiguration.gOnly.SetInspectionInputLink(GetCali
    RobotPose Moving_Pose = new RobotPose(m_Cave_Pose_Info[i].C
    // RobotPose Moving_Pose = new RobotPose(m_Cave_Pose_Info[i
    InspRunParams_Temp_Check[i] = new RunJobParameters(GetInspe
    InspRunParams_Temp_Check[i].Inputs["Input_Command"] = comma
    InspRunParams_Temp_Check[i].Inputs["CaveID"] = m_Cave_Pose_

    InspRunParams_Temp_Check[i].Inputs["MovingPoseX"] = m_Cave_
    InspRunParams_Temp_Check[i].Inputs["MovingPoseY"] = m_Cave_
    InspRunParams_Temp_Check[i].Inputs["MovingPoseDeg"] = CogMi

    InspRunParams_Temp_Check[i].Inputs["Mean"] = 255;

    if ((m_TerminalDatas_T2[m_Cave_Pose_Info[i].CaveID - 1] !=
    {
        InspRunParams_Temp_Check[i].Inputs["TerminalDatas_T2"]
        InspRunParams_Temp_Check[i].Inputs["TerminalDatas_T3"]
```

VisionTool Configuration - Inspection 5 - Insp 5 T4C Final Recheck

工具 输入/输出 图形

输入

名称	类型	值
MovingPoseX	System.Double	450.8
MovingPoseY	System.Double	58
MovingPoseDeg	System.Double	0
Input_Command	System.String	T4C,1,2018060...
AutoCom	System.Boolean	False
InputImage0	Cognex.VisionPro.CogImage8Grey	
InputImage0Home2D	Cognex.VisionPro.CogImage8Grey	Cognex.VisionP...
ProductName	System.String	Product 1
InputImageCameraSettings	System.String	Cam4_Exposur...
InputImage0CameraSettings	System.String	Cam4_Exposur...
InputImageLightChannels	System.String	
Mean	System.Double	255

输出

7.相机调用

相机的调用

当需要对相机进行特殊设置时候需要对相机进行调用，一般一般在“Prepare”中进行设置，不用写在Config中

m_framework.Cameras[0].FrameGrabber.OwnedGigEAccess.SetDoubleFeature("TriggerDelayAbs", Came1TriggerDelayAbs);

对该相机在（100,100）坐标取一个长800，宽600的像

```
public void Prepare()
{

    if (m_framework.Cameras.Count != 0 && m_framework.Cameras[0].AcqFifoOp != null)
    {
        m_framework.Cameras[0].AcqFifoOp.OwnedROIParams.SetROIXYWidthHeight(100, 100, 800, 600);
    }
    return ;
}

public ProfinetUserData ProfinetJobParser(CommandAndInfo
```

void ICogAcqROI.SetROIXYWidthHeight(int x, int y, int width, int height)
Sets the ROI.

异常:

8.标定模块的设置和添加。-参考第二阶段视频

9.设备映射逻辑介绍。-参考第二阶段视频

10.设备通讯逻辑介绍、-参考第二阶段视频

11.数据收集模块-参考第二阶段视频

12.跑图的设置。-参考第二阶段视频

三、Dataman 脚本调试与修改。

1.多码调试应用

Symbology Settings(码类型设置)：

General： 勾选需要解的码的类型

Multicode Settings(多码)： Number of codes 设置允许读取到的码的数量 适用于一幅图里有多个码

allow Partial Results： 勾选时可以将解出来的每一个码都输出

2.选型

扫码需求： 用于自动化设备还是手动工站

条码特性： 条码大小，规格，打码方式材料等

安装方式： 有多大空间可供扫码枪安装调试

公司主推产品： 如以前是 DM302X，现在是 DM262X 一次性扫一个码还是多个码等

常见固定式扫码枪型号： DM302X(停产)，DM262X-1540P，DM262X-1120/DM262X-1120F,DM50X,DM374X(大视野)

常见手持式扫码枪型号： DM8050HDXM，DM8600/DM8600 HDX

1.扫码枪选型：DM262x DM302x DM50x

参数	DataMan 50/60 Imager
图像传感器	Sensor 1/3 inch CMOS
图像传感器属性	4.51 mm x 2.88 mm (H x V), 6.0 µm square pixels
分辨率 (pixels)	752 x 480
快门速度	18 µs to 25 ms exposure
帧率	up to 60 fps at full resolution
镜头	45mm/70mm/110mm
光源	集成内部光源
尺寸	DM50 23.5mm x 26.5mm x 45.4mm DM60 55mm x 44.5mm x 23.5mm
通信方式	DM50 USB和RS-232 DM60 USB、RS-232和以太网
解码速率	最高解码速率为 45/秒

DM50x：

DataMan 262X 以太网读码器规格

图像分辨率	1280 x 960 全局快门
读取	45 fps
解码速率	45 次解码/每秒
镜头选择	6.2 mm（3 位镜头或液态镜头，40..200 mm） 16 mm（手动调焦或液态镜头，45 mm ..1 m）
照明	模块化照明/现场配置照明：四种独立控制，高功率 LED（红、白、蓝、IR）配有带通滤波器和偏光过滤器
通信	RS-232 和 以太网接口
尺寸	直立 - 43.1mm x 22.4mm x 64mm 折角 - 43.1 x 35.8mm x 49.3mm

DM262x：

DM262X 三个型号： DM262X（像素是 120W）用的都是液态镜头 可以自动对焦 302X 可以装液态和固态镜头

DM262X-1120-P 是白色透明盖子

DM262X-1120-F 黑色的盖子四个灯都是偏振光

DM262X-1540 黑色的镜头盖子 两个偏振光和强光

DataMan 302X 以太网读码器规格

图像分辨率	1280 x 1024
帧率	up to 60 fps
解码速率	45 次解码/每秒
镜头选择	10.3mm liquid lens 19 mm liquid lens 16mm/25mm lens
照明	Diffuse lens cover, red illumination (assembled) Polarized red LED high-powered integrated light 其他配件
快门速度	5us..1000us
通信	RS-232 和 以太网接口

DM302x：

- DM302X有八排灯，每一排都可以单独打开或者关闭；
- DM50X是固态镜头，不能自动对焦只能手动调焦距，跟视觉的相机一样，DM262X都是液态镜头可以自动对焦，DM302X可以装液态镜头也可以装固态镜头，看实际需求；
- DM262X用的是16mm的镜头，DM302X用的是10.5的镜头；
- DM262X像素是120W，DM302X像素是130W；
- DM8050X和DM8600X都是手持扫码枪，不用调试参数；

3.码密度计算

含义：码的每个 module(模块)的平均像素值 每个模块占了几个像素

意义：它是判断扫码稳定性的一个重要依据，它不是越大越好也不是越小越好，需要控制在一个合理的范围，二维码一般建议在 5~8，一维码建议在 3 以上

码PMM值计算

1.视野是78mm*50mm 中间有个二维码尺寸为5mm*5mm 二维码是12*12Code
(通俗来讲就是求此二维码每个模块包含几个像素)

用扫码枪DM50X(752*480)的长边计算

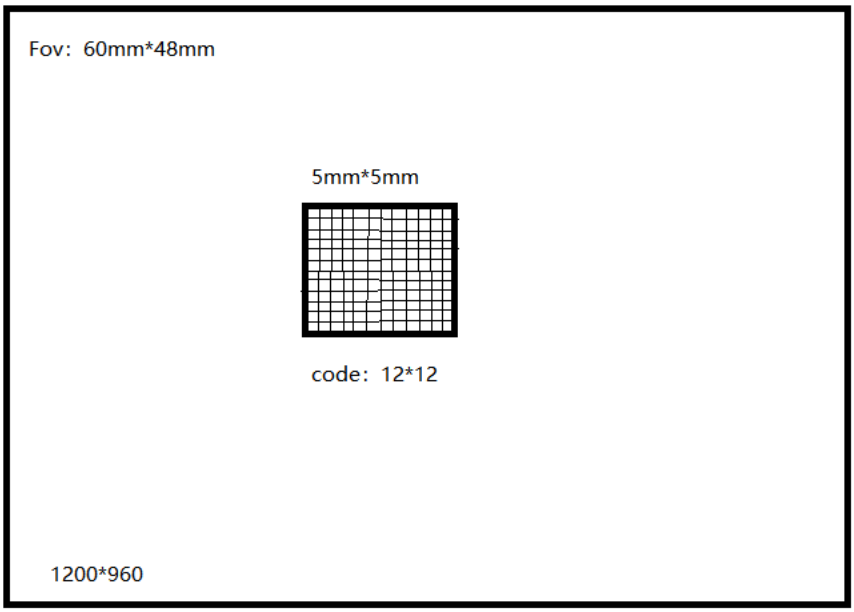
①先算出视野里每个mm单位所包含的像素个数:

752/78mm=9.6 表示每mm有9.6个像素

②再算出每个模块尺寸为多少mm: 5/12=0.4166666667mm

③计算码密度: 9.6*0.0.4166666667mm=4

另一种: 5mm/78mm*752像素/12=4.017094017094017094017094017094



黑色大框是扫码枪或者相机的视野: 60mm*48mm, 分辨率为1200*960

中间的小框是个二维码尺寸为: 5mm*5mm

这个二维码的码值为12*12, 也就是说二维码的是有144个模块儿(小格子)组成。

此时计算码密度:

1. 计算每个像素多长: 60mm/1200=0.05mm 这是每个小像素点的长宽

2.计算这个二维码长或宽占用了几个像素: 5mm/0.05mm=100

3.二维码长边有12个模块儿共计占用100个像素: 100/12=8.333

or:

(1200/12)/(5mm/60mm)=100/12=8.333

4. ID 脚本参考-3V 验证器评判标准

默认脚本

```
// Default script for data formatting
function onResult (decodeResults, readerProperties, output)
{
        if (decodeResults[0].decoded)
        {
        output.content = decodeResults[0].content;
        }
}
```

上图是默认脚本，意思是假如解到码了就输出这个条码

例子脚本1

```
// Default script for data formatting
function onResult (decodeResults, readerProperties, output)
{
        if (decodeResults[0].decoded)
        {
        output.content = decodeResults[0].content+'\r';
        }
}
```

上图是手持扫码枪使用的脚本，意思是假如解到码了就输出这个条码后并回车。

例子脚本2

```
// Default script for data formatting
function onResult (decodeResults, readerProperties, output)
{
  if (decodeResults[0].decoded)
    {
     output.content = "XYZ"+decodeResults[0].content+'\r\n';
    }
else
   {
    output.content = "NG";
   }
}
```

上图意思是假如解到码了就在条码前面加XYZ，后面跟回车换行，否则输出NG

COGNEX

例子脚本3

```
// Default script for data formatting
function onResult (decodeResults, readerProperties, output)
{
  if ((decodeResults[0].decoded&&decodeResults[0].content.length=="12")||
      (decodeResults[0].decoded&&decodeResults[0].content.length=="16"))
                {
                output.content = decodeResults[0].content+'\r';
                }
         else
         {
                output.content = "";
         }
}
```

上图意思是假如解到的码是12或16位，就输出并换行，否则就不输出

COGNEX

例子脚本4

```
function onResult (decodeResults, readerProperties, output)
{
        if ((decodeResults[0].decoded&&decodeResults[0].content.length=="17"))
                {
                  output.content = decodeResults[0].content.substr(0,1)
                }
        else
                {
                  output.content = "";
                }
}
```

上图意思是假如解到的码是17位，就输出第一位，否则就不输出

二.DataMatrix码的介绍

4.COGNEX-3V验证器评判标准

- COGNEX-3V是以AIM DPM指标作为评判标准，AIM DPM指标主要分为以下6项，如下：

① SymbolContrast：单元格对比度衡量的是明单元格与暗单元格之间的明暗差异。

② PrintGrowth：衡量的是印刷的墨水质量有无变化，导致墨点不一致。

③ UEC：衡量的是数据区域的损坏已经在多大程度上影响代码的读取安全边际。

④ Modulation：衡量的是一个模块是否有可能被错误地识别为明模块或暗模块。

⑤ Fixed Pattern Damage：衡量的是定位器图案、时钟图案和静音区的完整性。

⑥ GridNonUniformity：衡量的是符号网格交叉点偏离出理想位置的程度。

结论：
验证和可读性是不一样的
对于验证为A的DPM代码，使用低端的识读器也可能无法读取
对于验证为D的DPM代码，使用康耐视读码器和适当的光源可实现很高的读取率

0 | © 2020 Cognex Confidential

5.ID 知识

1.条码基础知识‘;

一维条码组成：静区 起始符 数据符 终止符 静区

二维条码组成：计时图案 数据区 模块或单元 L 型寻边区 静区

二维条码 DM 码 L 型寻边区对应的另一边是 时钟区：定义条码是 NxN 个模块组成

DataMatrix 码的判定：激光镭雕和喷墨的码质量需要检测：

Verifier：根据 ISO 15415 或 Aim Dpm 标准来定值

COGNEX-3V 验证器评判标准：ABCDE DE 不合格

常见不合格码种类：L 型寻边区缺失 对比度不佳 模块化不佳 模块偏离 条码损坏 条码变形(长宽非 1:1)

直接元件标识工艺： 四种方法：喷墨 打点阵和复刻 激光 电化蚀刻

一维二维区别：一维码存储数据少，尺寸太大 水平方向 破损不能读取

二维码存储数据多，尺寸小 水平加竖直方向 破损能读取

2.扫码枪软件介绍：

Displayed image settings 设置：Size：设置图片的质量 full 是原图 也可以是多少分之一

Image transfer settings 设置: 勾选 Transfer all images 可以使得当多通道存在时 每个通道解码结果显示一张图片

Symbology Settings(码类型设置)：

General：勾选需要解的码的类型

Multicode Settings(多码)： Number of codes 设置允许读取到的码的数量 适用于一幅图里有多个码

allow Partial Results：勾选时可以将解出来的每一个码都输出

Communication Settings(通信)：IP 设置 波特率设置

Custom Commands(字段)： 勾选(勾选后需设置内容 一般常见的+ — 或者 no off)或者不勾选 和机构对应(一般当收不到机构触发信号时优先考虑 IP 和这个触发 字段)

System Settings: 设置读取或未读取到 反馈的内容：read no read error -10000 等等

System Device log：当扫码枪报警灯亮 点这个 清除报警信息 可以解除报警

Live(实时)：勾选 Decoding 能够实时显示解码

勾选 Focus Feedback ，图像显示窗口右侧会显示一个带有颜色的计数条高度表示对焦情况，越低表示对焦越差

勾选 Automatic Exposure：表示自动曝光，增益滑动条 显示的是当前目标的像素灰度值

其中蓝框 ROI 区域：表示解码区域

Optimize Focus：优化对焦

Optimize Brightness：优化曝光

Symbology Assistant：码制助手

trigger type：触发模式

Train Code :训练条码

Advanced Trigger Settings：触发模式高级设置 和其他调节参数设置

训练界面参数介绍：

勾选 Tune Light Bank： 读码器会自动调节光源，如果我们清楚光源的设置，自动调节时，就会跳过光源的设定

勾选 Exhaustive Tuning：强制调节光源

当被禁用时：读码器一旦在某一光源设定下成功解码，将不再改变光源设置

当启用是：读码器会测试所有光源设置模式，无论是否已经解码成功， 训练完后会有三个选项选择：

勾选 Enable Fitter Tuning：DataMan 会使用图像过滤器

勾选：Optimize Focus During Tuning：训练时自动对焦

勾选 Train Code After Tuning：会在自动调节成功之后，对条码进行训练

触发模式参数含义：

Single：拍摄单张图片进行解码，每个 setup 会拍一张，可以设置单个 setup 的超时时间

Presentation：连续拍照，每次对视野中的单个条码进行解码

Manual：在足够长的时间里进行拍照，直到成功解码或者触发信号结束

Burst：拍摄一组图片解码，并在首次成功解码时停止解码，我们可以设定每组图片拍摄数量，还有每次拍照的时间间隔，每次拍照都可以设定一个超时时间

Self：与 Presentation 模式类似

Continuous：在触发信号结束之前连续拍照，可以设定每次拍照时间间隔

Timeout：表示读码器等待时间，知道读码器读码成功或者失败

Tune 的作用：读码器可以自动调节最佳光源亮度，自动调节最佳焦距，自动欧诺个调节最佳增益等信息，最后给出三种最优解码方案来选择

优点：

① 便于现场工程师快于调试

② 加快解码时间，提高解码成功率

③ 对于比较难读的码可以自动调节最佳解码方案

④ 便于现场零基础人员

Save Configuration：读取 保存调试好的读码参数 CFG 和 cdc 格式

调试流程： ①链接扫码枪

② 进入扫码枪软件

③ 根据条码情况，调整合适的视野尺寸和读码位置

优化曝光 频率 打光方式，选择需要的触发方式，保存并退出

④ 保存读码参数

IP：参考 CCD 的 IP 设置

端口：默认 23

4.扫码枪的脚本：

启用脚本格式：

例子：

5.PPM 值计算：

PPM 值的含义：码的每个 module 的平均像素值→→→→→其实就是 码里每个单元所占用的像素数

意义：判断扫码稳定性的一个重要依据，不是越大越好或者越小越好，需要控制在一个合理范围，二维码一般建议在 5~8，一维码建议在 3 以上

计算：例如 扫码枪 DM50x 分辨率为 752*480，视野 78mm*50mm，二维码尺寸 5mm*5mm，二维码模块数为 12*12，现在问二维码 PPM 值：即二维码每个模块占用的像素数

① 先计算每 MM 有多少个像素，用长边 752/78=9.6 每 mm 有 9.6 个像素

② 计算 二维码每个模块多少 mm：5/12= 0.41666667

③ 计算 PPM：9.6*0.41666667 =4 则每个模块占用 4 个像素

3.扫码枪多码调试应用

视野 50mm，50mm 5mm*5mm 12*12

机器视觉维护与应用技能等级要求--工作领域

一、机器视觉系统方案评估

1. 相机选型

选型有芯片类型、芯片尺寸、黑白彩色、曝光方式、帧率、接口、分辨率等依据

分辨率：通过视野大小和精度需求来确定 相机分辨率

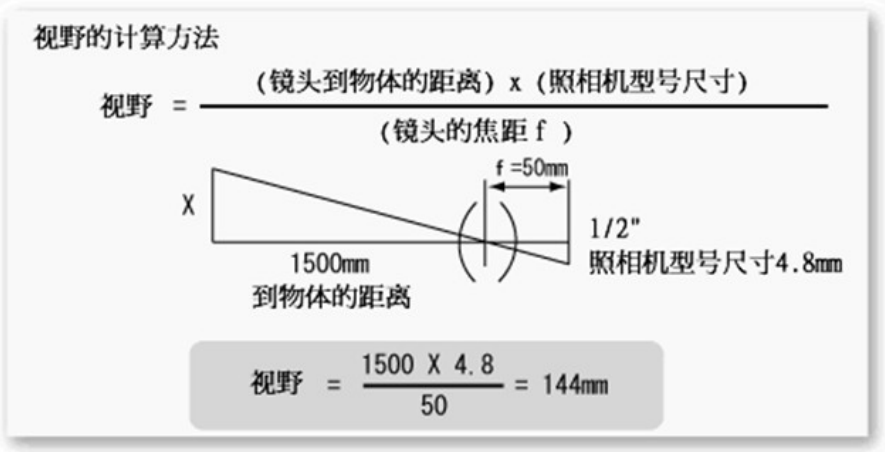
eg：视野(工件 10mm*8mm 允许 2mm 的浮动)：12mm*10mm，检测精度需求为 0.01mm 则理论需求的相机分辨率为：12mm/0.01mm=1200 以及 10mm/0.01mm=1000 此时选择一个分辨率略大于 1200*1000 的相机就能满足检测需求。

黑白彩色：一般不需要检测图像颜色信息的都用黑白相机
曝光时间：若是飞拍则需一定的最小曝光时间 原则是：最小曝光时间内物体或者相机移动的距离≤1 个像素就行。
帧率： 满足飞拍时连续拍照的时间间隔
数据接口：GigE cameLink USB
芯片尺寸：几分之一英寸
镜头接口：C 和 CS
快门方式：一般来说运动物体用全局快门，静止物体用卷帘快门

Cognex Cam-CIC-5000R-14-G：500w 卷帘快门 14 帧/秒 黑白相机
接口 GigE 分辨率 2592*1944 像素位深 12bits 靶面尺寸：2.5 分之一英寸
感光芯片：COMS 芯片尺寸：5.7mm*4.28mm

相机选型总结：
精度满足要求： 一般实际精度要高于项目所需求的精度 eg：需求为 0.02mm 我们可以给 0.01mm 甚至 0.005mm 的精度
确定色彩要求：
曝光时间：
帧率/数据接口：
芯片尺寸：
镜头接口：
其他相关

PS：常见芯片尺寸： ¼ 英寸为 3.6mm x 2.7mm 对角线 4mm
⅓ 英寸为 4.8mm x 3.6mm 对角线 6mm ½ 英寸为 6.4mm x 4.8mm 对角线 8mm
⅔ 英寸为 8.8mm x 6.6mm 对角线 11mm 1 英寸为 12.8mm x 9.6mm 对角线 16mm
物距*芯片尺寸=视野*焦距



例如： 工件尺寸为 10mm x 8mm 工作台上料有 2mm 误差 精度要求 0.02mm 工作距离 130mm
答：相机的视野需求最小也要 12mm x 10mm 实际可以给到比如 15mm x12mm 的视野大小（视野应略大于工件检测区域）
相机短边 12mm/0.02mm*⅓ =1800 相机长边 15mm/ 0.02mm*⅓=2250
Ps：这里给出的 0.02 精度又*⅓ 是根据项目需求不同 比如用作测量的项目要求精度更高此时*⅓ 可以给出更高的精度
此时可以选用 2590x1944 的 500w 分辨率的相机 黑白彩色根据实际需求
若是需要飞拍则考虑：快门方式为全局曝光 相机最小曝光时间（ms） 轴带着相机的运行速度或者是被拍摄物体运行的速度（mm/s）
某系统的拍摄精度是 0.1mm/像素，相机曝光时间是 1/2000 秒，拍摄物体运动速度是 10mm/s，这样目标在曝光时间内物体运动的距离是 0.005mm<<0.1mm，因此可以用该系统拍摄。
这里曝光是 0.0005s 速度是 10mm/s 距离则是 10*0.0005=0.005mm 这个距离小于给出的拍摄精度 0.1mm 则代表这个拍摄系统可以用于此工作场景

2. 镜头选型

有接口、焦距、倍率、工作距离、支持的最大分辨率等依据

① 根据相机芯片大小和工作空间限制确定使用镜头的焦距或放大倍率 **物距*芯片尺寸=视野*焦距**
② 考虑是否需要远心镜头
③ 景深是否满足需求
④ 镜头是否兼容相机芯片尺寸：应大于等于
⑤ 确定镜头分辨率：应大于等于相机分辨率
⑥ 确定畸变率是否满足要求
⑦ 超大视野或超小视野
⑧ 是否需要考虑透过光谱
⑨ 镜头是否要配合其他配件
⑩ 价格等
奥普特镜头：

OPT-AC2514-5M：A:奥普特 C：C 型接口 25：焦距 25mm 14：F 值为 1.4 5M:5 百万像素
OPT-AC5024-2M：A:奥普特 C：C 型接口 50：焦距 50mm 24：F 值为 2.4 2M:2 百万像素
OPT-AM1-110：1：放大倍率：1 倍 110：工作距离 110mm
放大倍率： 像距/物距 芯片尺寸/视野

镜头命名规则：

CCTV 镜头：OPT-AC2514-5M
OPT：表示奥普特
AC：表示 C 接口定焦镜头
25：焦距为 25mm
14：F 值为 1.4
5M：5 百万像素
OPT-AC5024-2M：奥普特、C 接口定焦镜头、焦距 50MM、F 值为 2.4、2 百万像素
远心镜头：OPT-AM1-110：
奥普特 1：表示放大倍率　　110：表示工作距离 110MM
放大倍率=像距/物距=芯片尺寸/视野
假如此相机芯片尺寸为 1/3 英寸即 4.8mm*3.6mm
此时焦距：f=物距 x 芯片尺寸/视野长边=130x4.8/15=41.6mm
此时根据焦距 41.6mm 以及 500w 分辨率选用合适的镜头（镜头分辨率应大于等于相机分辨率）
镜头的最大兼容芯片尺寸必须大于相机芯片尺寸（简单理解为假如相机芯片为 1mmx1mm 尺寸的话 镜头直径必须大于 1mm 此时镜头才不会遮挡相机视野）

镜头和相机要考虑接口类型是否匹配：
C 型镜头匹配 C 型相机　CS 型镜头匹配 CS 型相机
C 型镜头+5mm 接圈匹配 CS 型相机　CS 型镜头不匹配 C 型相机

3. 光源选型：

有光源本身的颜色、形状、发光类型等，产品的表面材质反光特性、光源安装高度和照射角度等依据

方向： 暗场 亮场 无影光 漫射背光 平行背光 根据检测物体表面是否反光或者凹凸不平等

低角度光(暗场)：主要用于边缘有倒角、圆角物体轮廓提取、冲压、浇筑、浮雕图案识别与检测，光滑表面　划伤、裂痕检测。

缺点： 对于透明物体表面的划痕检测效果不理想效果不理想

高角度光(亮场)：主要用于表面粗糙程度不同区域的区分、边缘或内部有垂直断差或者比较陡峭（超过 60 度） 边缘检测或测量。

缺点： 对于反光区域相差不大的效果不理想

背光：用于轮廓和边缘检测；用于透明物体内不透明物体的检测。

非同轴漫射光应用及光路图：可以避免因弯曲表面导致的打光不均匀

同轴光： 含有 50%度银镜。同轴光源能够凸显物体表面不平整，克服表面反光造成的干扰，主要用于检测　物体平整光滑表面的碰伤、划伤、裂纹和异物。

光谱：可见光 近可见光 不可见光　根据黑白彩色相机 物体颜色 等

其他：偏振片(偏振光):用于减少眩光或者是镜面反射　其它配件

彩色光源：黑白相机对 660nm 波长的红光最敏感 且红色 LED 成本低 较常使用

蓝色光：波长短，适合检测物体表面质量
紫光：波长更短，散射性好
白色：中性颜色，适合拍彩色图片，或者被摄物体颜色有变化，
绿色：亮度高且波长和蓝色接近，可替代蓝色
红外：用于透明物体检测，波长越长穿透力越强，波长越短，扩散性越好

照射光的种类： 平行光 直射光 散射光 偏光光
平行光：照射角度整齐的光，太阳光就是平行光。发光角度越窄的 LED 直射光越接近平行光。
直射光：LED 光源直接照射对象物的光
散射光：各种角度的光源混合在一起的光，日常生活用光几乎都是扩散光。

偏光光：光源的传递方向在特定的垂直平面上波动收到限制的光。通常是利用偏光板来防止特定方向的反射

选型思路：

① 了解项目需求，明确要检测或者测量的目标
② 分析目标与背景的区别，找出两者之间最可能差异大的光学现象
③ 根据光源与目标之间的配合关系，初步确定光源的发光类型
④ 拿实际光源测试，以确定满足要求的打光方式
⑤ 根据具体确定适用于客户的产品

具体选型：

① 考虑方向选用： 直射光 漫射光
② 考虑颜色 可见光 近可见光 不可见光
③ 偏振片 等其它配件
④ 考虑选用：漫射背光 平行背光 暗场配光 明场配光 无影光

1)背光源：用作物体轮廓提取、透明物体内不透明物体的检测、贯穿性缺陷检测、狭缝或通孔内杂质检测 透明物体表面划痕、内部异物、破损等检测

2)Dome 光：用做物体表面弯曲 凹凸不平整 球面 等场景

3）条光：多打侧面角度光 条码识别，定位标记识别、液晶元件检测、大面积物体表面划痕检测、零部件边缘检测

4）环光：产品包装外观、标签检测、显微镜照明、通用外观检测 5)

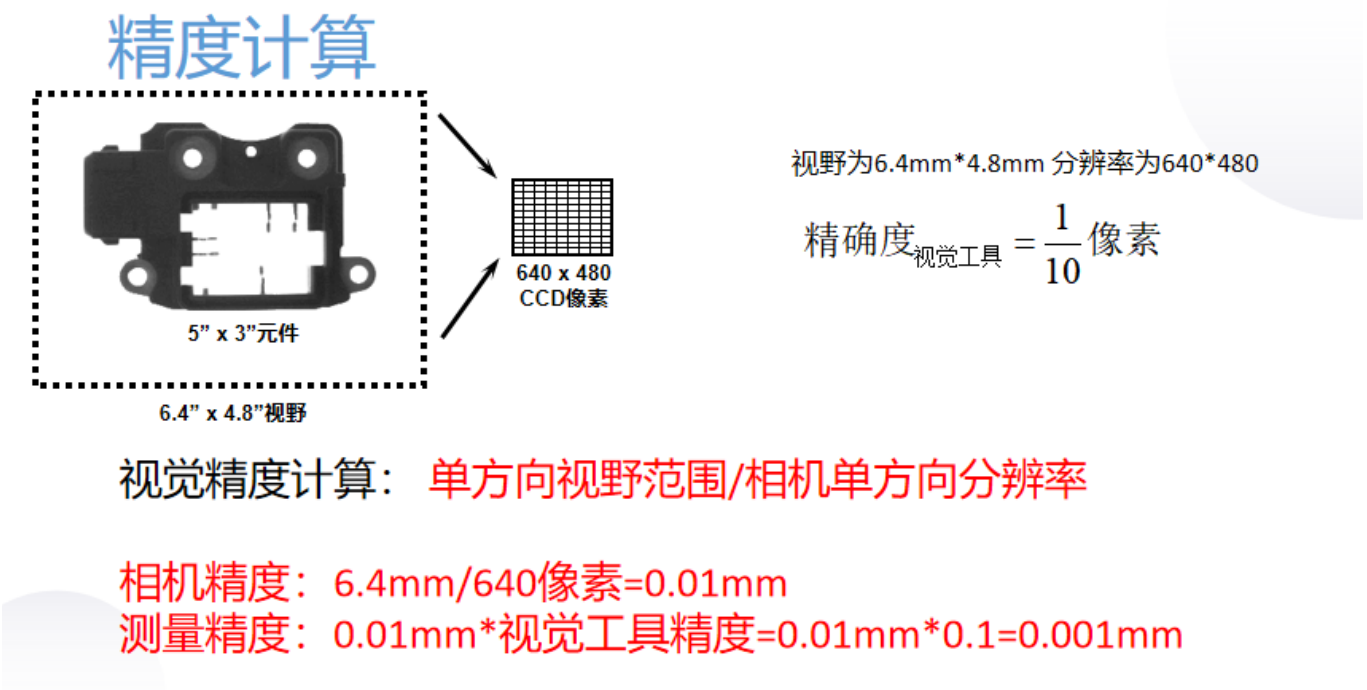
5）同轴光：金属玻璃瓶等高反光表面的划痕检测、晶片上激光标注检测、轴承、饮料罐上刻印字符

光源控制器选择：

1.普通光源控制器 光源常亮 亮度一般 用做定拍场景

2.高亮频闪光源控制器 光源不是常亮 亮度高 一般用作飞拍场景

4.精度计算



eg： 1.康耐视 500w 相机拍照，视野为 50mm*40mm，所使用的视觉工具精度为 ¼ 个像素，求测量精度？（500w 相机分辨率为 2592*1944）

相机精度:(即像素分辨率) 相机精度=50mm/2592=0.0193mm

测量精度：测量精度=相机精度*视觉工具精度=0.0193mm*¼=0.004825mm

3D 视觉系统图像处理

图像获取：Xscale、StepsPerLine、DistancePerCycle、MeasuringFiled 参数设定

XScale(mm)：X 方向的分辨率 意味着 X 方向采集一张图片的宽度为多少多少 mm，

StepsPerLine：采集一个像素所需要的脉冲数

DistancePerCycle：指的是脉冲信号一圈下来的距离 多少个脉冲信号走一圈是根据接的触发线来决定

比如 DS910B UVFlex 项目 A+A-B+B-四个线则表示每四个脉冲信号走一圈，

MeasuringFiled：相机视野大小，不同的视野对应着不同的采集频率，决定了采集区域和采集速度

曝光时间：0.1-1ms

eg:常见康耐视 3D 相机型号：DS925B(0.003)、 DS910B(0.002)

DS910B:四触发线 A+A-B+B- 对应粉灰紫黑四个颜色

例 1:

XScale=0.04mm(单帧图像的宽度)

StepsPerLine=20 个脉冲(1 帧图像需要的脉冲数)

DistancePerCycle=0.008mm(电机走一圈的距离)

MeasuringFiled=7

上述参数设置意为：20 个脉冲信号对应四个触发线，此时电机走五圈，一圈 0.008mm，五圈下来的距离为 0.04mm

MeasuringFiled=7 表示相机此时取像速度为 552Hz(等同 552 帧/s)

例 2:

XScale=0.04mm

StepsPerLine=20 个脉冲

DistancePerCycle=0.008mm

MeasuringFiled=7

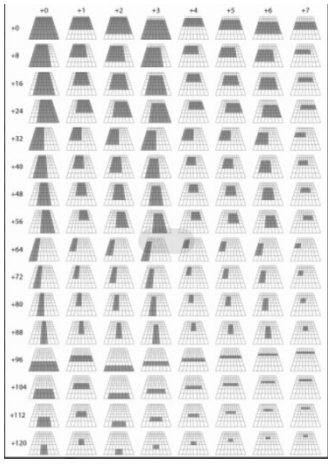
此时已知测量长度为 9.2mm 问带着相机的轴的轴速最大为多少?

单张图片宽度 0.04mm，求得测量长度为 9.2mm 时需要采集：9.2mm/0.04mm=230 张图片，

当前参数下帧率为 552/s，则测完此长度需要的时间：230/552=0.41666667s

一秒钟测量的距离：9.2mm/0.41666667s =22mm/s

理论最大运动速度为 22mm/s



Maximum Acquiring Rate Depending on the Measuring Field Number								
	+0	+1	+2	+3	+4	+5	+6	+7
0	144Hz	192Hz	192Hz	192Hz	284Hz	284Hz	284Hz	552Hz
8	181Hz	239Hz	239Hz	239Hz	354Hz	354Hz	354Hz	680Hz
16	181Hz	239Hz	239Hz	239Hz	354Hz	354Hz	354Hz	680Hz
24	181Hz	239Hz	239Hz	239Hz	354Hz	354Hz	354Hz	680Hz
32	241Hz	318Hz	318Hz	318Hz	469Hz	469Hz	469Hz	892Hz
40	241Hz	318Hz	318Hz	318Hz	469Hz	469Hz	469Hz	892Hz
48	241Hz	318Hz	318Hz	318Hz	469Hz	469Hz	469Hz	892Hz
56	241Hz	318Hz	318Hz	318Hz	469Hz	469Hz	469Hz	892Hz
64	362Hz	476Hz	476Hz	476Hz	694Hz	694Hz	694Hz	1200Hz
72	362Hz	476Hz	476Hz	476Hz	694Hz	694Hz	694Hz	1200Hz
80	362Hz	476Hz	476Hz	476Hz	694Hz	694Hz	694Hz	1200Hz
88	362Hz	476Hz	476Hz	476Hz	694Hz	694Hz	694Hz	1200Hz
96	552Hz	552Hz	1030Hz	1030Hz	1030Hz	1030Hz	1030Hz	1030Hz
104	680Hz	680Hz	1200Hz	1200Hz	1200Hz	1200Hz	1200Hz	1200Hz
112	892Hz	892Hz	1200Hz	1200Hz	1200Hz	1200Hz	1200Hz	1200Hz
120	1200Hz	1200Hz	1200Hz	1200Hz	1200Hz	1200Hz	1200Hz	1200Hz

左图为 MeasuringFiled 参数不同数值对应的视野范围及景深

右图为 MeasuringFiled 参数不同数值对应的取像速度

4.3D 相机相关问题讲解 DS925B DS910B

Xscale、StepsPerLine、DistancePerCycle、MeasuringFiled 等参数解释

以 XY 方向一帧一帧的扫码图像，最终拼成一幅图片

测高原理则是 三角测量法：

超时：超时时间设置应大于相机采集图像所需的时间

参数解释： 触发方式为硬件半自动

图像属性：宽度 高度 等于采出来的图 的长宽尺寸

自定义属性：例如 XScale=0.04mm，StepsPerLine=20 个脉冲 DistancePerCycle=0.008mmMeasuringFiled=7

XScale(mm)：X 方向的分辨率 意味着 X 方向采集一张图片的宽度为多少多少 mm，

StepsPerLine：采集一个像素所需要的脉冲数

DistancePerCycle：指的是脉冲信号一圈下来的距离 多少个脉冲信号走一圈是根据接的触发线来决定

比如 DS910B UVFlex 项目 A+A-B+B-四个线则表示每四个脉冲信号走一圈，

MeasuringFiled：相机在 Intensity 模式下，的一个视野大小，不同的视野对应着不同的采集频率，决定了采集区域和采集速度

曝光时间：0.1-1ms

5.光源安装位置设计

能够根据工作场景要求正确设定光源安装高度和角度

6. 相机、镜头安装位置设计

能够根据工作场景要求正确设计相机安装高度和角度

7. 项目风险评估

能够根据机构精度、安装空间、物料特征、成本、产能等因素，综合评估项目潜在风险

8. 确认视觉方案文档

能够根据项目评估情况，确定软件版本、硬件选型、打光方案、工控机性能要求、项目风险、通讯协议、BOM，编写视觉方案文档

二、视觉系统集成与应用

1 视觉系统集成

2.1.1 能够正确分析项目需求，并根据项目需求编写视觉方案

2.1.2 能够根据项目需求完成相机、镜头与光源的选型

2.1.3 能够根据工作任务完成视觉系统硬件组装

2.1.4 能够根据工作任务完成打光测试与验证

2 视觉系统编程

2.2.1 能够根据工作任务和设备机构布局搭建视觉系统集成方案

2.2.2 能够根据工作任务和设备机构布局验证视觉系统集成方案

2.2.3 能够根据视觉系统集成方案编写视觉应用程序

2.2.4 能够根据工作任务完成视觉系统与设备通讯的联机测试

3 视觉系统调试

2.3.1 能够根据批量生产环境，对视觉系统工具、通讯、标定等各项参数予以调校和测试

2.3.2 能够验证视觉系统程序兼容性和稳定性